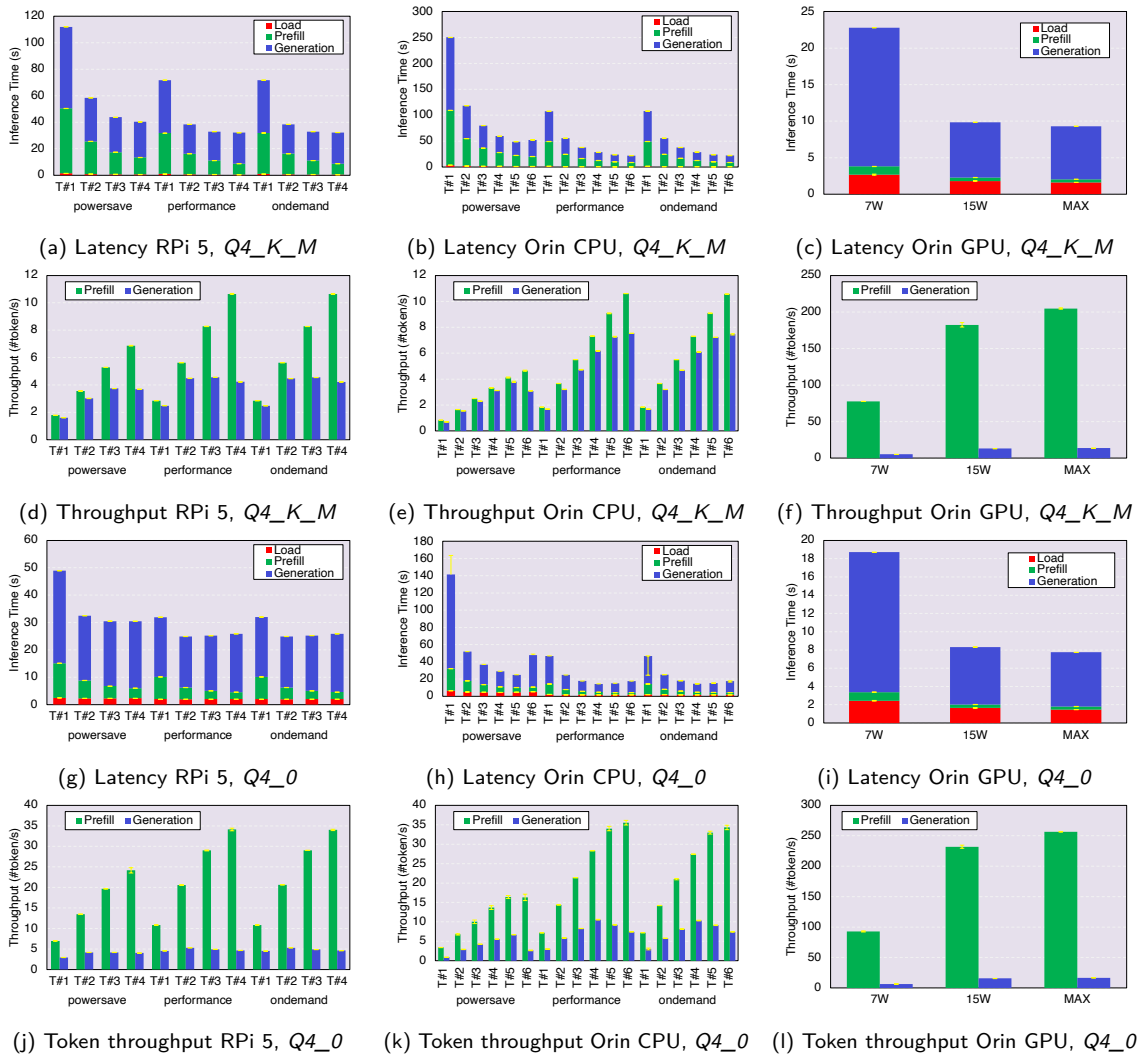


(a) Latency RPi 5, *Q4_K_M* (b) Latency Orin CPU, *Q4_K_M* (c) Latency Orin GPU, *Q4_K_M*

(d) Throughput RPi 5, *Q4_K_M* (e) Throughput Orin CPU, *Q4_K_M* (f) Throughput Orin GPU, *Q4_K_M*

(g) Latency RPi 5, *Q4_0* (h) Latency Orin CPU, *Q4_0* (i) Latency Orin GPU, *Q4_0*

(j) Token throughput RPi 5, *Q4_0* (k) Token throughput Orin CPU, *Q4_0* (l) Token throughput Orin GPU, *Q4_0*

Fig. 5. Latency and throughput for multiple configurations, all quantization schemes and all devices

An exception was observed with the *powersave* governor on the Orin, where the six-thread configuration resulted in a performance drop compared to five threads. This behavior was not observed on the RPi 5 but appears to be linked to a significantly higher number of context switches during the generation phase in this configuration, as illustrated in Figure 7. On the Orin GPU, the 15W power mode more than halves end-to-end latency compared to 7W mode, with maximum GPU frequency providing further minor latency improvements. When analyzing time to first token (TTFT)—the sum of load latency and prefill latency—distinct patterns emerge based on the quantization scheme. For *Q4_K_M* quantization, CPU inference is dominated by the prefill phase, which accounts for over 90% of TTFT across all configurations. In contrast, for GPU inference, the load phase is the bottleneck, making up 80% of TTFT. This is due to

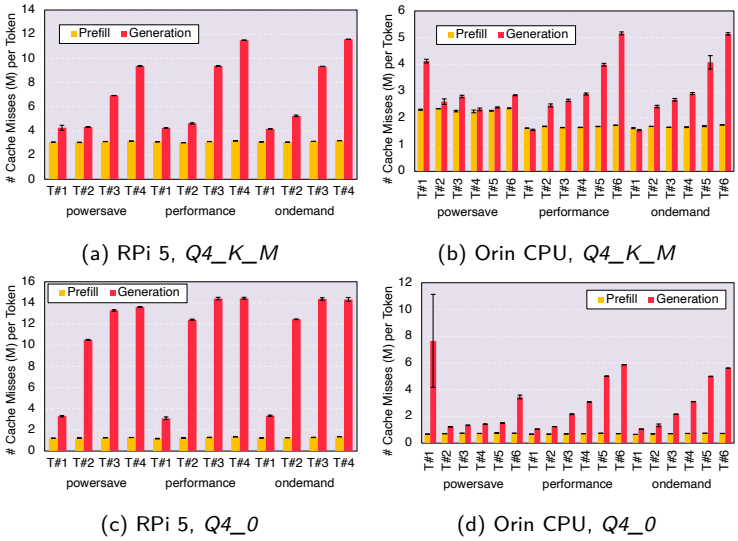
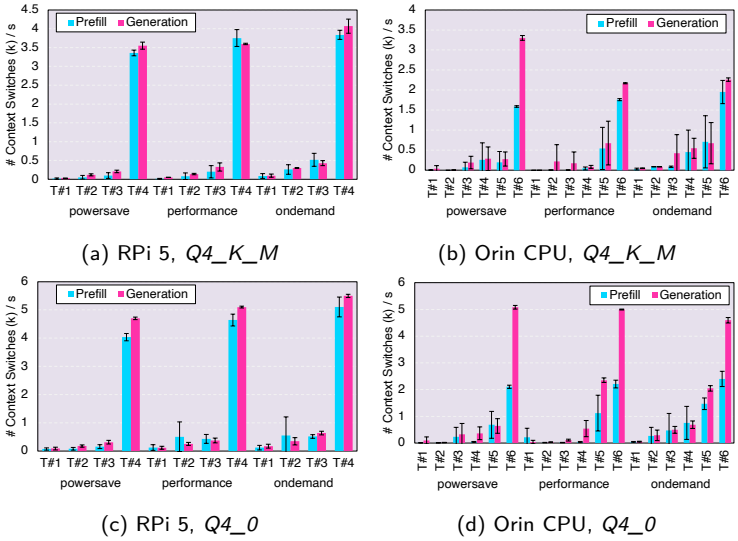


Fig. 6. Level-1-cache misses for multiple configuration, all quantization schemes, RPi 5, and Orin CPU

two factors: *(i)* the prefill phase is significantly shorter on the GPU than on the CPU, and *(ii)* he load phase takes 2–2.5 seconds on the GPU, compared to 1 second for CPU inference. For *Q4_0* quantization, the CPU inference load phase accounts for a significantly higher portion of TTFT (43%) in the fastest configuration. This is due to a substantial increase in loading time, which quadrupled in the worst case compared to *Q4_K_M* quantization, depending on the configuration. Additionally, *Q4_0* provides drastically lower prefill latencies, which were reduced by a factor of 3 to 4. For GPU inference, *Q4_0* quantization reduced both load and prefill latencies. However, the reported short load times are only valid for runs 2–5, as the model remains cached in memory between runs. In first-run scenarios, much higher load times were observed, ranging from 5 seconds to over 200 seconds, depending on the model size, quantization scheme, and device. However, the first runs simulate a cold-start scenario, in which the model is only loaded from scratch upon the first arriving request. In a real-world scenario these long load times could be omitted by pre-loading and caching the model in memory.

As shown in Figure 5, *Q4_0* quantization more than tripled prefill throughput and increased generation throughput by $\sim$ 10–70% for CPU inference, compared to *Q4_K_M* quantization. This improvement aligns with expectations, as *Q4_0* quantization incorporates Arm processor optimizations applied during the load phase [30]. Interestingly, GPU inference also exhibited performance gains, varying strongly between 3% for Llama 3.1 8B and 380% for Yi 1.5 6B in the prefill phase and between 1% for Flan T5 Large and 30% for Gemma 2 9B for the generation phase depending on the model and power mode. The huge prefill improvements of Yi 1.5 6B can be explained by the lower memory usage of the model which eliminates the memory saturation effect already discussed earlier. However, for most setups the performance improvement lay between 20% and 30 % in the prefill phase and in the generation phase most smaller models saw a throughput improvement of 5-10%, while for larger models it typically lay in the range from 15% to 20%.



(a) RPi 5, *Q4_K_M* (b) Orin CPU, *Q4_K_M*

(c) RPi 5, *Q4_0* (d) Orin CPU, *Q4_0*

Fig. 7. Context switches for multiple configuration, all quantization schemes, RPi 5, and Orin CPU

For CPU inference, prefill phase throughput scales roughly linearly with increasing thread count, with *performance* and *ondemand* governors yielding higher throughputs than the *powersave* governor. Similarly, generation throughput also increases linearly with thread count but eventually plateaus. On the Orin, plateauing occurs at 6 threads while, on the RPi 5, plateauing occurs at 3 or 4 threads with *Q4_K_M* quantization, depending on the CPU governor. This behavior was observed for all models except Flan T5 Large, which did not exhibit a clear performance plateau. For the *powersave* governor, 3 threads yielded the highest throughput, while for *performance* and *ondemand* governors, 2 and 3 threads performed similarly. With *Q4_0* quantization, 2 and 3 threads also performed similarly under powersave, whereas 2 threads provided the highest throughput for the other two governors. On the Orin CPU with *Q4_0* quantization, a decline in prefill throughput was observed at 6 threads, while generation throughput peaked at 4 or 5 threads, depending on the model, before decreasing at higher thread counts under the *performance* and *ondemand* governors. For the *powersave* governor, 5 threads provided the best generation throughput. The decline in generation performance at higher thread counts is likely due to the generation phase being memory-bound, where performance is primarily constrained by memory access latency rather than computational power [45]. As a result, increasing compute resources beyond a certain point has only minimal impact on throughput. As observed in our results, increasing the thread number beyond the optimum can even be harmful to performance due to the overhead of saving and restoring the processor state during context switches caused by time-sharing of CPU-cores among the threads. Additionally, background processes competing for CPU-time may displace the llama.cpp threads and cause additional context switches. This effect was observed in our experiments, which is evident from the exploding number of context switches with maximum thread count visible in Figure 7. Context switches can, moreover, negatively impact cache performance and, therefore, increase average memory access times, as data used by one thread may replace data cached by the previously running thread and, therefore, cause additional cache misses [34]. This effect was also found